

EC-333 Microprocessor and Interfacing Techniques

Experiment # 5

Procedure and Stacks

1. Objective/s:

- Procedure
- Stack Operations
- Example Codes
- Lab Task

2. Procedures

In a procedural environment, the basic unit of code is the *procedure*. A procedure is a set of instructions that compute some value or take some action (such as printing or reading a character value). The definition of a procedure is very similar to the definition of an *algorithm*. A procedure is a set of rules to follow which, if they conclude, produce some result. An algorithm is also such a sequence, but an algorithm is guaranteed to terminate whereas a procedure offers no such guarantee.

2.1. Near/Far Procedure

The 80x86 supports near and far subroutines. Near calls and returns transfer control between procedures in the same code segment. Far calls and returns pass control between different segments. The two calling and return mechanisms push and pop different return addresses. You generally do not use a near call instruction to call a far procedure or a far call instruction to call a near procedure. Given this little rule, the next question is “how do you control the emission of a near or far call or ret?” Most of the time, the call instruction uses the following syntax:

call ProcName

and the ret instruction is:

ret

or *ret disp*

Unfortunately, these instructions do not tell MASM if you are calling a near or far procedure or if you are returning from a near or far procedure. The proc directive handles that chore. The proc directive has an optional operand that is either near or far. Near is the default if the operand field is empty³. The assembler assigns the procedure type (near or far) to the symbol. Whenever MASM assembles a call instruction, it emits a near or far call depending on operand. Therefore, declaring a

symbol with `proc` or `proc near`, forces a near call. Likewise, using `proc far`, forces a far call. Besides controlling the generation of a near or far call, `proc`'s operand also controls `ret` code generation. If a procedure has the near operand, then all return instructions inside that procedure will be near. MASM emits far returns inside far procedures.

2.2. Syntax

2.2.1. Procedure Declaration

- The syntax of procedure declaration is the following:
- **PROC name type ; body of procedure ret ENDP name**
- type can be **NEAR** (in same segment) or **FAR** (in a different segment) -- if omitted, **NEAR** is assumed

2.2.2. Procedure Calling

- `CALL` invokes a procedure
- `CALL` has two forms,
- **direct :**
 - call *name* (where **name** is the name of a procedure)
- **indirect:**
 - call *address_expression* (where **address_expression** specifies a register or memory location containing the address of a procedure)

2.2.3. Return Instruction

- To return from a procedure, the instruction is executed
 - **ret pop_value**
- The integer argument **pop_value** is optional
- **ret** causes the stack to be popped into IP
- If **pop_value N** is specified, it is added to SP -- in effect removes *N* additional bytes from the stack

3. Stacks

During interrupt and subroutine operations, the contents of specific internal registers of the 8086 may be overwritten. If these registers contain data that are needed after the return, they should be PUSHed to a section of memory known as the Stack. Here they may be maintained temporarily. At the completion of the service routine or subroutine, these values are POPped off the stack in the reverse order of the PUSH, and the register contents are restored with their original data.

When an interrupt occurs, the 80x86 automatically PUSHes the current flags, the value in CS, and the value in IP onto the stack. As part of the service routine for the interrupt, the contents

of other registers may be pushed onto the stack by executing PUSH instructions. An example is the instruction PUSH SI. It causes the contents of the Source Index register to be PUSHed onto the stack.

At the end of the service routine, POP instructions can be included to restore the values from the stack back into their corresponding internal registers. For example, POP SI causes the value at the top of the stack to be popped back into the source index register.

As indicated earlier, stack is implemented in the memory of the 8086. It is a maximum of 64K bytes long and is organized as 32K words. The lowest addressed byte in the current stack is pointed to by the contents of the stack segment (SS) register.

3.1. Stack Segment

Any number of stacks may exist in an 8086. A new stack can be brought in by simply changing the value in the SS register through software. For instance, executing the instruction MOV SS,DX loads a new value from DX into SS. Even though many stacks can exist, only one can be active at a time.

3.2. Stack Pointer

Another register, the stack pointer (SP) contains an offset of the current top of the stack from the value in SS. The address obtained from the contents of SS and SP is the physical address of the last storage location in the stack to which data were PUSHed. This is known as the top of the stack. The value in the stack pointer starts at 0FFFFh upon initialization of the 8086. Combining this value with the current value in SS gives the highest addressed location in the stack: that is, the bottom of the stack.

Since data transfers to and from stack are always 16-bit words, it is important to configure the system such that all stack locations are at even word boundaries. This minimizes the number of memory cycles required to PUSH or POP data for the stack and minimizes the amount of time required to perform a switch in program context.

The 8086 PUSHes data and addresses to the stack one word at a time. Each time a register value is to be PUSHed onto the top of the stack, the value in the stack pointer is first decremented by 2 and then the contents of the register are written into memory. In this way, we see that the stack grows down in memory from the bottom of the stack, which corresponds to the physical address derived from SS and 0FFFFh toward the end of the stack, which corresponds to the physical address obtained from SS and offset 000016.

When a value is popped from the top of the stack, the reverse of this sequence occurs. The physical address defined by SS and SP always points to the location of the last value pushed onto the stack. Its contents are first popped off the stack and put into the specified register within the 8086; then SP is incremented by 2. The top of the stack now corresponds to the previous value pushed onto the stack.

4. Explanation

There are a few things that we must remember are:

- We must make sure to create a Stack, and make it large enough for any program that has a CALL or any program that will use the PUSH and POP instructions.
- PUSH all registers that contain data or addresses that we need after the RETURN
- When we RETURN we must POP the registers in the reverse order of the PUSH
- We must POP each register that we PUSH
- If we PUSH before the CALL, we must POP after the RETURN (in the calling module or subroutine)
- If we PUSH after the CALL, we must POP before the RETURN (inside the subroutine)

PUSH Examples

PUSH AX – the contents of a 16-bit register

PUSH EBX - the contents of a 32-bit register

PUSHA (286 and higher) – Preserves all usable registers of 80286

PUSHAD (386 and higher) – Preserves all usable registers of 80386

There are corresponding POP instructions for each of the above examples. Execution of a PUSH instruction causes the data corresponding to the operand to be pushed onto the top of the stack.

For instance, if the instruction is **PUSH AX** its execution results in the following:

```

SP <- SP - 1      ;SP is decremented
SS:SP <= AH       ;AH is PUSHed on the Stack
SP <- SP - 1      ;SP is decremented
SS:SP <= AL       ;AL is PUSHed on the Stack
  
```

This shows that the two bytes of AX are saved in the stack part of memory and the stack pointer is decremented by 2 such that it points to the new top of the stack. On the other hand, if the instruction is **POP AX**, its execution results in the following:

```

AL <- SS:SP       ;AL is POPped from the Stack
SP <- SP + 1      ;SP is incremented
AH <= SS:SP       ;AH is POPped from the Stack
SP <- SP + 1      ;SP is incremented
  
```

Let us assume we wish to write a program that CALLs a subroutine that will need all the general purpose registers, and those registers contain data that we will need after the subroutine has finished. We may preserve the contents of the registers using the PUSH instruction either by:

PUSHing before the CALL and POPping after the RETurn

PUSHing after the CALL and POPping before the RETurn

Regardless of which method we use, we must POP in the reverse order of the PUSH. Consider the following.

```
PUSH AX
PUSH BX
PUSH CX
PUSH DX
CALL SUB1
POP DX
POP CX
POP BX
POP AX
```

The above code will preserve the registers on the Stack, and execute the subroutine, which PUSHes the IP onto the Stack. The last instruction of the subroutine is the RETurn which POPs the contents of the Stack into the IP, and then we restore the registers with the POP instruction. The above code will perform this correctly. Notice the register operand of the first POP corresponds to the register operand of the last PUSH, while the register operand of the last POP corresponds to the register operand of the first PUSH.

It is incorrect to perform the reverse. Consider the following.

```
PUSH AX
PUSH BX
PUSH CX
PUSH DX
CALL SUB1
POP AX
POP BX
POP CX
POP DX
```

The previous code **INCORRECTLY** restores the registers. The contents of the AX register before the CALL is restored to DX. This code Swaps registers as follows

The contents of the AX and DX registers are swapped and the contents of the BX and CX registers are swapped. It is difficult to say what effect this might have on your program, but if the data in any of these registers are used, errors could be significant.

A disastrous scenario is if we PUSH outside the subroutine and POP outside the subroutine, or PUSH inside the subroutine and POP outside the subroutine. This causes a register content to be placed in the IP. We have no idea where our program will continue, but we should expect the program to crash.

5. Tasks

1. Write a program in which all general purpose registers are utilized. During program call a procedure in which again all four general purpose registers are used. But before jumping to procedure save all registers contents by using stack operations and after returning from stack, retain the registers values. Use simple arithmetic operations.
2. Write a program that virtually create three different stacks. With SP= 1000; SP=2000; SP=3000 Push first 10 odd numbers to the first stack, first ten even numbers to the second stack and write all ones to the last stack.
3. Create two stacks, swap the data of both the stacks
4. Write a program to reverse the stored string using stack.